CSE 2201 / 2202

Lab Practice Programs

C++ source collection

11 programs, in sequence

Each program starts on a new page. Long programs continue on the next page.

01 0_1_knapsack.cpp

02 Backtracking.cpp

03 Connected Component.cpp

04 Disjoint Kruskal.cpp

05 Disjoint Set Cycle.cpp

06 Floyd Warshall.cpp

07 Fractional Knapsack.cpp

08 Maximum Sum Subarray.cpp

09 Segment Tree.cpp

10 lcs.cpp

11 lis.cpp

01. 0_1_knapsack.cpp

Source file: 0_1_knapsack.cpp

```cpp
#include <algorithm>
#include <iostream>
using namespace std;
int main()
{
    int n, w;
    cout << "Enter the number of items and the maximum weight: ";
    cin >> n >> w;

    int weight[101], value[101];

    for (int i = 1; i <= n; i++)
    {
        cout << "Enter weight and value for item " << i << ": ";
        cin >> weight[i] >> value[i];
    }

    int dp[101][101] = {0};

    for (int i =  1; i <= n; i++)
    {
        for (int j = 0; j <= w; j++)
        {
            if(weight[i] <= j)
            {
                dp[i][j] = max(dp[i - 1][j],
                               value[i] + dp[i - 1][j - weight[i]]);
            }
            else
            {
                dp[i][j] = dp[i - 1][j];
            }
        }
    }

    cout << dp[n][w] << endl;
    int j = w;

    for(int i = n; i >= 1; i--)
    {
        if(dp[i][j] != dp[i - 1][j])
        {
            cout << "item" << i << endl;
            j -= weight[i];
        }
    }
}
```

02. Backtracking.cpp

Source file: Backtracking.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int n;
vector<vector<int>> board;

bool isSafe(int row, int col)
{
    //column check
    for(int i = 0; i < row; i++)
    {
        if(board[i][col] == 1)
        {
            return false;
        }
    }

    // Right diagonal check
    for(int i = row -1, j = col + 1; i >=0 && j < n; i--, j++)
    {
        if(board[i][j] == 1)
        {
            return false;
        }
    }

    // Left diagonal check
    for(int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
    {
        if(board[i][j] == 1)
        {
            return false;
        }
    }

    return true;
}

bool placeQueen(int row)
{
    if(row == n)
    {
        return true;
    }

    for(int col = 0; col < n; col++)
    {
        if(isSafe(row, col))
        {
            board[row][col] = 1;

            if(placeQueen(row + 1))
            {
                return true;
            }

            board[row][col] = 0;
        }

    }

    return false;
}

```

02. Backtracking.cpp (continued)

Source file: Backtracking.cpp

```
67
68
69  int main()
70  {
71
72      cout << "Enter the queen number: "<< endl;
73      cin >> n;
74      board.resize(n, vector<int>(n, 0));
75
76      bool result = placeQueen(0);
77
78      if(result)
79      {
80          for(int i = 0; i < n; i++)
81          {
82              for(int j = 0; j < n; j++)
83              {
84                  cout << board[i][j] << " ";
85              }
86              cout << endl;
87          }
88      }
89      else
90      {
91          cout << "NO Solution";
92      }
93
94      return 0;
95  }
```

03. Connected Component.cpp

Source file: Connected Component.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

vector<int> graph[100]; // Store the neighbors of vertex i. Also called adjacencey list.
bool visited[100];

void dfs(int node)
{
    visited[node] = true;

    for(int next : graph[node])
    {
        if (!visited[next])
        {
            dfs(next);
        }
    }
}


int main()
{
    int v, e;
    cout << "Number of Verticees and Number of Edges: ";
    cin >> v >> e;

    for(int i = 0; i < e; i++)
    {
        int u, w;
        cout << "\nEnter the endpoint of edges: ";
        cin >> u >> w;

        graph[u].push_back(w);
        graph[w].push_back(u);
    }

    int components = 0;

    for(int i = 0; i < v; i++)
    {
        if(!visited[i])
        {
            dfs(i);
            components++;
        }
    }

    cout << "Connected Components: " << components << endl;

    return 0;
}
```

04. Disjoint Kruskal.cpp

Source file: Disjoint Kruskal.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> parent;
vector<int> ranki;

// Sort edges by weight
bool sortbywt(const tuple<int, int, int>& a, const tuple<int, int, int>& b)
{
    return get<2>(a) < get<2>(b);
}

// Find the root of a vertex
int findop(int u)
{
    if(parent[u] == -1)
    {
        return u;
    }

    return parent[u] = findop(parent[u]);
}

// Union two sets
void unionop(int u, int v)
{
    if(ranki[u] > ranki[v])
    {
        parent[v] = u;
    }
    else if(ranki[v] > ranki[u])
    {
        parent[u] = v;
    }
    else
    {
        parent[u] = v;
        ranki[v]++;
    }
}

int main()
{
    int v, e;
    cin >> v >> e;

    // Store edges: source, destination, weight
    vector<tuple<int, int, int>> edges;

    for(int i = 0; i < e; i++)
    {
        int sc, des, wt;

        cin >> sc >> des >> wt;

        edges.push_back(make_tuple(sc, des, wt));
    }

    // Sort edges by increasing weight
    sort(edges.begin(), edges.end(), sortbywt);

    // Initially, every vertex is a separate set
    parent.assign(v, -1);
    ranki.assign(v, 0);

    // Store MST edges
```

04. Disjoint Kruskal.cpp (continued)

Source file: Disjoint Kruskal.cpp

```
67      vector<tuple<int, int, int>> mst;
68
69      long long totalWeight = 0;
70
71      // Go through edges from smallest weight to largest
72      for(const auto& edge : edges)
73      {
74          int sc = get<0>(edge);
75          int des = get<1>(edge);
76          int wt = get<2>(edge);
77
78          // Find roots
79          int rootsc = findop(sc);
80          int rootdes = findop(des);
81
82          // If roots are different, adding this edge will not create a cycle
83          if(rootsc != rootdes)
84          {
85              unionop(rootsc, rootdes);
86
87              mst.push_back(edge);
88
89              totalWeight += wt;
90
91              // MST of v vertices always has v - 1 edges
92              if((int)mst.size() == v - 1)
93              {
94                  break;
95              }
96          }
97      }
98
99      // Check whether an MST was successfully created
100     if(v > 0 && (int)mst.size() == v - 1)
101     {
102         cout << "Edges in the MST:" << endl;
103
104         for(const auto& edge : mst)
105         {
106             cout << get<0>(edge) << " "
107                  << get<1>(edge) << " "
108                  << get<2>(edge) << endl;
109         }
110
111         cout << "Total Weight: " << totalWeight << endl;
112     }
113     else
114     {
115         cout << "The graph is disconnected: No MST exists" << endl;
116     }
117
118     return 0;
119 }
```

05. Disjoint Set Cycle.cpp

Source file: Disjoint Set Cycle.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

vector<int> parent;

int find_op(int u)
{
    if(parent[u] == -1)
    {
        return u;
    }
    return find_op(parent[u]);
}

void union_op(int root1, int root2)
{
    parent[root2] = root1;
}

bool is_cycle(vector<pair<int, int>> edges)
{
    for(auto edge : edges)
    {
        int src = edge.first;
        int des = edge.second;

        int src_root = find_op(src);
        int des_root = find_op(des);

        if(src_root == des_root)
        {
            return true;
        }

        union_op(src_root, des_root);
    }
    return false;
}

int main()
{
    int v = 5;

    vector<pair<int, int>> edges =
    {
        {0, 1},
        {0, 2},
        {1, 3},
        {2, 4},
        {1, 4}
    };

    parent.resize(v, -1);

    if(is_cycle(edges))
    {
        cout << "Cycle exist" << endl;
    }
    else
    {
        cout << "No Cycle" << endl;
    }

    return 0;
```

05. Disjoint Set Cycle.cpp (continued)

Source file: Disjoint Set Cycle.cpp

```
67  }
```

06. Floyd Warshall.cpp

Source file: Floyd Warshall.cpp

```cpp
#include <iostream>
#include <algorithm>


using namespace std;

int main()
{
    int n;
    cout << "Number of Vertices: ";
    cin >> n;

    int dist[101][101];

    cout << "Enter the adjacency matrix row by row.\n";
    cout << "For each row, enter " << n << " values (edge weights).\n";

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cin >> dist[i][j];
        }
    }

    for(int k = 0; k < n; k++)
    {
        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < n; j++)
            {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }

    cout << "\nShortest Distance Matrix:\n";


    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cout << dist[i][j] << " ";
        }

        cout << endl;
    }

    return 0;
}
```

07. Fractional Knapsack.cpp

Source file: Fractional Knapsack.cpp

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

struct Item
{
    int weight;
    int value;
};


bool compare(Item a, Item b)
{
    return (double)a.value / a.weight > (double)b.value / b.weight;
}


int main()
{
    int n, W;
    cin >> n >> W;

    Item item[100];

    for(int i = 0; i < n; i++)
    {
        cin >> item[i].weight >> item[i].value;
    }

    sort(item, item + n, compare);

    double total = 0;

    for(int i = 0; i < n; i++)
    {
        if(item[i].weight <= W)
        {
            W = W - item[i].weight;
            total = total + item[i].value;
        }
        else
        {
            total = total + ((double)item[i].value / item[i].weight) * W;
            break;
        }
    }
    cout << "Maximum value: " << total << endl;

    return 0;
}
```

08. Maximum Sum Subarray.cpp

Source file: Maximum Sum Subarray.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int divideConquer(vector<int> a, int b, int e)
{
    // Base Case
    if (b == e)
    {
        return a[b];
    }

    int mid = (b + e) / 2;

    // Left half

    int leftsum = divideConquer(a, b, mid);

    // Righ half
    int rightsum = divideConquer(a, mid + 1, e);

    // Cross Sum

    int leftMax = a[mid];
    int total = 0;

    for(int i = mid; i >=b; i--)
    {
        total = total + a[i];

        if (total > leftMax)
        {
            leftMax = total;
        }
    }

    int rightMax = a[mid + 1];
    total = 0;

    for (int i = mid + 1; i <= e; i++)
    {
        total = total + a[i];

        if(total > rightMax)
        {
            rightMax = total;
        }
    }

    int crossSum = leftMax + rightMax;

    return max({leftsum, rightsum, crossSum});

}

int main()
{
    vector<int> a = {2, 3, -8, 7, 2, -1, 3};

    int n = a.size();

    cout << "Array: ";
    for (int x: a)
    {
```

08. Maximum Sum Subarray.cpp (continued)

Source file: Maximum Sum Subarray.cpp

```
67            cout << x << " ";
68        }
69        cout << endl;
70
71        cout << "Maximum Sum: "<< divideConquer(a, 0, n - 1);
72
73        return 0;
74    }
```

09. Segment Tree.cpp

Source file: Segment Tree.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

vector<int> tree;

// Build
void build(int node, int l, int r, vector<int>& arr)
{
    // Base
    if (l == r)
    {
        tree[node] = arr[l];
        return;
    }

    int mid = (l + r) / 2;

    build(2 * node + 1, l, mid, arr);

    build(2 * node + 2, mid + 1, r, arr);

    tree[node] = tree[2 * node + 1] + tree[2* node + 2];
}

// range sum
int query(int node, int l, int r, int ql, int qr)
{
    // NO overlap
    if (l > qr || r < ql)
    {
        return 0;
    }
    // Complete overlap
    if (l >= ql && r <= qr)
    {
        return tree[node];
    }
    // Partial overlap

    int mid = (l + r) / 2;

    int leftsum = query(2 * node + 1, l, mid, ql, qr);

    int rightsum = query(2 * node + 2, mid + 1, r, ql, qr);

    return leftsum + rightsum;
}

//point update

void update(int node, int l, int r, int index, int value)
{
    if (l == r)
    {
        tree[node] = value;
        return;
    }

    int mid = (l + r) / 2;

    if (index <= mid)
    {
        update(2 * node + 1, l, mid, index, value);
    }
    else
```

09. Segment Tree.cpp (continued)

Source file: Segment Tree.cpp

```
67      {
68          update(2 * node + 2, mid + 1, r, index, value);
69      }
70
71      tree[node] = tree[2* node + 1] + tree[2 * node + 2];
72  }
73
74
75
76  int main()
77  {
78      vector<int> arr = {2, 1, 5, 3, 4, 7, 6, 8};
79      int n = arr.size();
80
81      tree.assign(4 * n, 0);
82      build(0, 0, n - 1, arr);
83
84      cout << "Range Sum (0 - 5):" << query(0, 0, n - 1, 0, 5) << "\n";
85
86      update(0, 0, n - 1, 2, 10);
87
88      cout << "Range Sum (0 - 5):" << query(0, 0, n - 1, 0, 5) << "\n";
89
90  }
```

10. lcs.cpp

Source file: lcs.cpp

```cpp
#include <iostream>
#include <algorithm>
#include <string>
using namespace std;

int main()
{
    string X, Y;
    cout << "Enter the strings: ";
    cin >> X >> Y;

    int m = X.length();
    int n = Y.length();

    int dp[101][101] = {0};

    // Build DP table
    for (int i = 1; i <= m; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (X[i - 1] == Y[j - 1])
            {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else
            {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    // Find the actual LCS
    string lcs = "";

    int i = m;
    int j = n;

    while (i > 0 && j > 0)
    {
        if (X[i - 1] == Y[j - 1])
        {
            lcs = X[i - 1] + lcs;
            i--;
            j--;
        }
        else if (dp[i - 1][j] > dp[i][j - 1])
        {
            i--;
        }
        else
        {
            j--;
        }
    }

    cout << "Length: " << dp[m][n] << endl;
    cout << "LCS: " << lcs << endl;

    return 0;
}
```

11. lis.cpp

Source file: lis.cpp

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

int main()
{
    int n;
    cout << "Number of Element: "<< endl;
    cin >> n;

    int a[100];
    int dp[100];

    for(int i = 0; i < n; i++)
    {
        cout << "Element "<< i << endl;
        cin >> a[i];
        dp[i] = 1;
    }

    for(int i = 1; i < n; i++)
    {
        for(int j = 0; j < i; j++)
        {
            if(a[j] < a[i])
            {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
    }

    int answer = 0;

    for(int i = 0; i < n; i++)
    {
        answer = max(answer, dp[i]);
    }

    cout<< "LIS length: "<< answer << endl;

    return 0;
}
```